FALL SEMESTER 2025-2026

VITYARTHI PROJECT

NAME: KRITTIKA HALDAR

REGISRATION NUMBER: 25BAS10019

CLASS CODE: CSA2001

Predicting Prices of Car

Table Of Content

1. Project Overview and Details
2. Data Collection and Acquisition
3. Modules Used
4. Visualization
5. Steps Of Program Functions
6. Results

Predicting Prices of Car

Project Overview And Details

1. Title and Goal of Project

Title: Used Car Price Segment Prediction and Mileage Trend Analysis

Goal: The primary goal was to develop a machine learning model capable of classifying the price of a used car into predefined segments ('Low', 'Medium', or 'High'). Additionally, the project aimed to conduct Exploratory Data Analysis (EDA) to visualize and understand the average car mileage trends across different manufacturing years.

2. Business Value and Application

This project delivers high business value, primarily in the used car market ecosystem**:**

- Pricing Strategy for Dealerships: Provides automated, data-driven categorization of inventory, helping dealers quickly estimate a car's fair price segment. This improves pricing efficiency and profit margins.
- Risk and Inventory Management: By segmenting cars into price brackets, businesses can better manage inventory turnover and assess the market risk associated with different vehicle types.
- Consumer Insight: Buyers and sellers on online platforms can use this model to set realistic expectations for a car's price based on its characteristics (year, mileage, engine power, etc.).
- Market Trend Analysis: The mileage trend analysis helps manufacturers and resellers understand shifts in vehicle efficiency over time, informing future purchasing or design decisions.

Predicting Prices of Car

3. Methodology Snapshot

Section	Description
Data Source	Car_Details.csv (Features include year, km driven, fuel type, engine, max power, etc.)
Pre-Processing	**Cleaning:** Standardized and converted units for mileage, engine (CC), and max_power (bhp). Imputed missing values using the median. **Feature Engineering:** Extracted car_brand from name and calculated car_age.
Target Transformation	The continuous selling_price was transformed into a categorical target (Low, Medium, High) using quantile-based binning to enable classification modeling.
Model Used	Random Forest Classifier (A robust ensemble method suitable for multi-class classification tasks).
Evaluation Metrics	**Classification:** Accuracy, Precision, Recall, F1 Score. **Regression Context:** R^2 and RSME
Visualization	A **Confusion Matrix** was used to visualize the model's classification performance, showing correct and incorrect predictions across the three price ranges.

4. Key Performance Highlights (Test Set)

The model was tested on an unseen subset of the data (Test Set) to evaluate its generalization capability. The results for the classification task are as follows:

Predicting Prices of Car

- Accuracy Score : 0.8679
- Precision Score : 0.8711
- Recall Score : 0.8679
- F1 Score : 0.8689
- R^2 : 0.9734
- Root Mean Squared Error : $141,037.63

Prediction Example

- Sample Car Input: Maruti Swift Dzire VDI, 2018
- Predicted Price Range: High

Data Collection and Acquisition

This section outlines the origin and characteristics of the dataset used to train the Real Estate Price Prediction model.

1. Source Data

The entire project is built upon the data contained in the file **"Car_Details.csv"**. This dataset is a version of the widely used , which is a highly referenced benchmark for regression tasks in machine learning.

2. Dataset Size and Scope

The dataset contained the following dimensions before initial cleaning:

- Number of Records (Rows): Approximately 8,128 car listings.
- Number of Features (Columns): 13 distinct variables.
- Time Period: The car models listed span across several years, providing a wide chronological range for trend analysis**.**

3. Key Variables Acquired

The acquired dataset included a comprehensive set of features relevant for predicting a car's market value and price segment. These features are critical for both the classification model and the mileage trend analysis:

Feature Name	Description	Data Type Acquired	Relevance to Goal
name	Full model name of the car (e.g., Maruti Swift Dzire VDI).	Text/String	Used to derive the car_brand.
year	Manufacturing year of the car.	Integer	Used to calculate car_age and analyze mileage trends.

Predicting Prices of Car

selling_price	The target price in Indian Rupees (₹).	Numeric	The variable converted to the categorical target (Low, Medium, High).
km_driven	Total kilometers the car has traveled.	Integer	Key predictor of wear and tear, and market value.
fuel	Type of fuel used (e.g., Diesel, Petrol, CNG).	Categorical	Influences running cost and price.
seller_type	Whether the seller is an Individual or a Dealer.	Categorical	Affects pricing strategy.
transmission	Type of transmission (Manual or Automatic).	Categorical	Influences market demand and price.
owner	Number of previous owners (e.g., First Owner, Second Owner).	Categorical	Key determinant of car quality and price.
mileage	Fuel efficiency (e.g., 23.4 kmpl).	Text/String	Required cleaning to extract numeric value; used for trend analysis.
engine	Engine displacement (e.g., 1248 CC).	Text/String	Key performance indicator.
max_power	Maximum engine power (e.g., 74 bhp).	Text/String	Key performance indicator.
torque	Engine torque and RPM information.	Text/String	Dropped due to complexity and high correlation with max_power.
seats	Number of seating capacity.	Integer	Basic feature of the car's physical configuration.

Predicting Prices of Car

Modules Used

The project was executed using the following core Python libraries, categorized by their primary function:

1. **Data Manipulation and Numerical Operations**

- pandas: The fundamental library for data manipulation, cleaning, and preparation. It was used to load the Car_Details.csv file, handle missing values, perform data type conversions (e.g., extracting numbers from text), and manage features.
- numpy: Used for advanced numerical computing, particularly for calculating the square root in RMSE (Root Mean Squared Error) and for general array operations required by machine learning models.

2. **Machine Learning and Modeling (Scikit-learn)**

All modeling and metric calculations were handled by the comprehensive scikit-learn (sklearn) library:

- sklearn.model_selection.train_test_split: Used to partition the dataset into separate training and testing sets, ensuring the model is evaluated on unseen data.
- sklearn.preprocessing.StandardScaler: Applied to standardize (scale) numerical features, preventing features with larger values (like km_driven) from dominating the model training process.

Predicting Prices of Car

- sklearn.ensemble.RandomForestClassifier: The core algorithm used. This ensemble classifier was trained to predict the car's price range (Low, Medium, High).
- sklearn.metrics: A crucial sub-module used to calculate all performance metrics:
    - Classification Metrics: accuracy_score, precision_score, recall_score, and f1_score.
    - Regression Metrics (for context): r2_score and mean_squared_error (used to derive RMSE).
    - Visualization Helper: confusion_matrix (used to generate the data for the Confusion Matrix graph).

3. **Data Visualization**

- matplotlib.pyplot: The foundational plotting library in Python. It was used to customize and display the final graph outputs.
- seaborn: A high-level data visualization library based on matplotlib. It was used specifically to create the aesthetically pleasing Confusion Matrix heatmap for the classification evaluation.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier # Switched to Classifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score # Need these for the requested R2/RMSE context
```

Visualization

The visualization module served two critical purposes: evaluating the performance of the machine learning model (Classification) and uncovering market trends from the raw data (Exploratory Data Analysis).

1. Confusion Matrix (Model Evaluation)

Purpose: To visually assess the performance of the **Random Forest Classifier** in segmenting car prices into the three categories (Low, Medium, High).

Description: The Confusion Matrix is a heatmap that compares the actual (true) price range of the cars in the test set against the price range predicted by the model.

- **Axes:**
  - **Y-axis (Actual Class):** The true segment of the car (e.g., truly 'High' priced).
  - **X-axis (Predicted Class):** The segment predicted by the classification model.
- **Interpretation (Key Insight):**
  - The highest values are concentrated along the main diagonal, confirming a high rate of **correct classifications** (True Positives).
  - The model achieved a high Accuracy, Recall, and F1 Score (all around 87%), indicating it correctly segments the price categories most of the time.

2. Average Mileage Trend Line Graph (Exploratory Data Analysis)

Purpose: To analyze how the average fuel efficiency (mileage) of cars in the dataset has changed over the manufacturing years.

Predicting Prices of Car

Description: This is a **Line Graph** that plots the average mileage (in kmpl) calculated for each manufacturing year present in the dataset.

- **Axes:**
  - **X-axis:** Manufacturing Year (ranging from 1994 to 2020).
  - **Y-axis:** Average Mileage (kmpl).
- **Interpretation (Key Insight):**
  - The line graph generally shows an **increasing trend** in average mileage over time. This reflects the automotive industry's continuous focus on improving engine efficiency and fuel economy through newer technologies and stricter emission norms.
  - This trend analysis is valuable for understanding long-term market shifts in consumer preferences toward fuel-efficient vehicles.

Predicting Prices of Car

Steps of Program Functions

1. Loading Data

```
# Load the dataset
df = pd.read_csv("/content/drive/MyDrive/Car_Details.csv")
```

2. Data Cleansing

```
# --- 1. Data Cleaning and Preprocessing ---
def clean_and_convert(df, column, unit_to_remove):
    df[column] = df[column].astype(str).str.replace(unit_to_remove, '', regex=False).str.strip().replace('null', np.nan)
    return df

# Apply cleaning steps
df['mileage'] = pd.to_numeric(df['mileage'].astype(str).str.split(' ').str[0].replace('null', np.nan), errors='coerce')

df = clean_and_convert(df, 'engine', 'CC')
df['engine'] = pd.to_numeric(df['engine'], errors='coerce')

df = clean_and_convert(df, 'max_power', 'bhp')
df['max_power'] = pd.to_numeric(df['max_power'], errors='coerce')

# Drop complex/unnecessary column
df.drop('torque', axis=1, inplace=True)

# Fill missing values (using direct assignment to avoid FutureWarning)
df['mileage'] = df['mileage'].fillna(df['mileage'].median())
df['engine'] = df['engine'].fillna(df['engine'].median())
df['max_power'] = df['max_power'].fillna(df['max_power'].median())

df.dropna(inplace=True)

# Feature Engineering
df['car_brand'] = df['name'].apply(lambda x: x.split(' ')[0])
df.drop('name', axis=1, inplace=True)

owner_map = {'First Owner': 1, 'Second Owner': 2, 'Third Owner': 3, 'Fourth & Above Owner': 4, 'Test Drive Car': 5}
df['owner'] = df['owner'].map(owner_map)

current_year = 2025
df['car_age'] = current_year - df['year']
df.drop('year', axis=1, inplace=True)
```

3. Feature Engineering

```python
# --- 2. TARGET VARIABLE TRANSFORMATION (Classification) ---
# We convert the continuous 'selling_price' into a discrete 'price_range' (Low, Medium, High)
# This allows us to calculate F1-score, Precision, Recall, and Accuracy.
df['price_range'] = pd.qcut(df['selling_price'], q=3, labels=['Low', 'Medium', 'High'])

# Store original selling_price for R2/RMSE context
y_regression = df['selling_price']
```

4. Encoding And Scaling

```python
# --- 3. Encoding and Scaling ---
categorical_cols = ['fuel', 'seller_type', 'transmission', 'car_brand']
numerical_cols = ['km_driven', 'mileage', 'engine', 'max_power', 'seats', 'owner', 'car_age']

# Encode all categorical features
df_encoded = pd.get_dummies(df.drop('selling_price', axis=1), columns=categorical_cols, drop_first=True)

# Define features (X) and new target (y)
X = df_encoded.drop('price_range', axis=1)
y_classification = df_encoded['price_range']

# Scale numerical features
scaler = StandardScaler()
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])
```

5. Train Model

```python
# --- 4. Model Training (Classification) ---
X_train, X_test, y_train, y_test = train_test_split(X, y_classification, test_size=0.2, random_state=42)

# Initialize and train the Random Forest Classifier
clf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1, max_depth=15, min_samples_split=5)
clf_model.fit(X_train, y_train)

# Make classification predictions
y_pred_clf = clf_model.predict(X_test)
```

6. Evaluation (Classification)

```python
# --- 6. Regression Metrics Context (Using the original model from previous step for context) ---
X_reg_train, X_reg_test, y_reg_train, y_reg_test = train_test_split(X, y_regression, test_size=0.2, random_state=42)
reg_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1, max_depth=15, min_samples_split=5) # Keeping the CLASSIFIER for prediction (as per user request context)
reg_model.fit(X_reg_train, y_reg_train)
y_pred_reg = reg_model.predict(X_reg_test)


rmse = np.sqrt(mean_squared_error(y_reg_test, y_pred_reg))
r2 = r2_score(y_reg_test, y_pred_reg)
```

7. Prediction Function

```
# --- 7. Prediction Function (Predicts Range) ---
# Use the CLASSIFIER model for prediction
def predict_car_range(model, input_data, scaler, df_columns, numerical_cols):
    # Transformation steps are the same as before
    input_df = pd.DataFrame([input_data])
    input_df['car_brand'] = input_df['name'].apply(lambda x: x.split(' ')[0])
    input_df['car_age'] = current_year - input_df['year']
    input_df['owner'] = input_df['owner'].map(owner_map)

    input_df.drop(['name', 'year', 'torque'], axis=1, inplace=True, errors='ignore')
    input_df = pd.get_dummies(input_df, columns=categorical_cols, drop_first=True)

    # Align features and fill missing with 0
    final_input_df = input_df.reindex(columns=X.columns, fill_value=0)

    # Scale numerical features
    final_input_df[numerical_cols] = scaler.transform(final_input_df[numerical_cols])

    predicted_range = model.predict(final_input_df)[0]
    return predicted_range

# Example Usage: User Input Simulation (Predicts Range)
sample_car_input = {
    'name': 'Maruti Swift Dzire VDI',
    'year': 2018,
    'km_driven': 30000,
    'fuel': 'Diesel',
    'seller_type': 'Individual',
    'transmission': 'Manual',
    'owner': 'First Owner',
    'mileage': 25.0,
    'engine': 1248,
    'max_power': 74.0,
    'seats': 5,
    'torque': '190Nm@ 2000rpm'
}

df_columns = X.columns
predicted_range = predict_car_range(clf_model, sample_car_input, scaler, df_columns, numerical_cols)
```

8. Printing Results

```
# --- 8. Print ONLY requested results and generate Confusion Matrix ---

print("\n--- Model Evaluation (Classification) and Prediction Results ---")
print("NOTE: We converted the price prediction to a 'Low/Medium/High' classification problem")
print("to calculate the requested Classification Metrics (F1, Precision, Recall, Accuracy).")
print("-" * 75)

print(f"Accuracy Score: {accuracy:.4f}")
print(f"Precision Score (Weighted): {precision:.4f}")
print(f"Recall Score (Weighted): {recall:.4f}")
print(f"F1 Score (Weighted): {f1:.4f}")

print("\n--- Regression Metrics (For Context, calculated using classification setup) ---")
print(f"R-squared (R2 Score): {r2:.4f}")
print(f"Root Mean Squared Error (RMSE): ${rmse:,.2f}")

print(f"\nPredicted Price Range for 'Maruti Swift Dzire VDI, 2018': {predicted_range}")
print("-" * 75)
```

9. Vizualization

```
# --- 9. Visualization (Confusion Matrix for Classification) ---
cm = confusion_matrix(y_test, y_pred_clf, labels=['Low', 'Medium', 'High'])
cm_df = pd.DataFrame(cm, index=['Actual Low', 'Actual Medium', 'Actual High'],
                     columns=['Predicted Low', 'Predicted Medium', 'Predicted High'])

plt.figure(figsize=(8, 6))
sns.heatmap(cm_df, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix for Car Price Range Classification')
plt.ylabel('Actual Class')
plt.xlabel('Predicted Class')
plt.show()
```

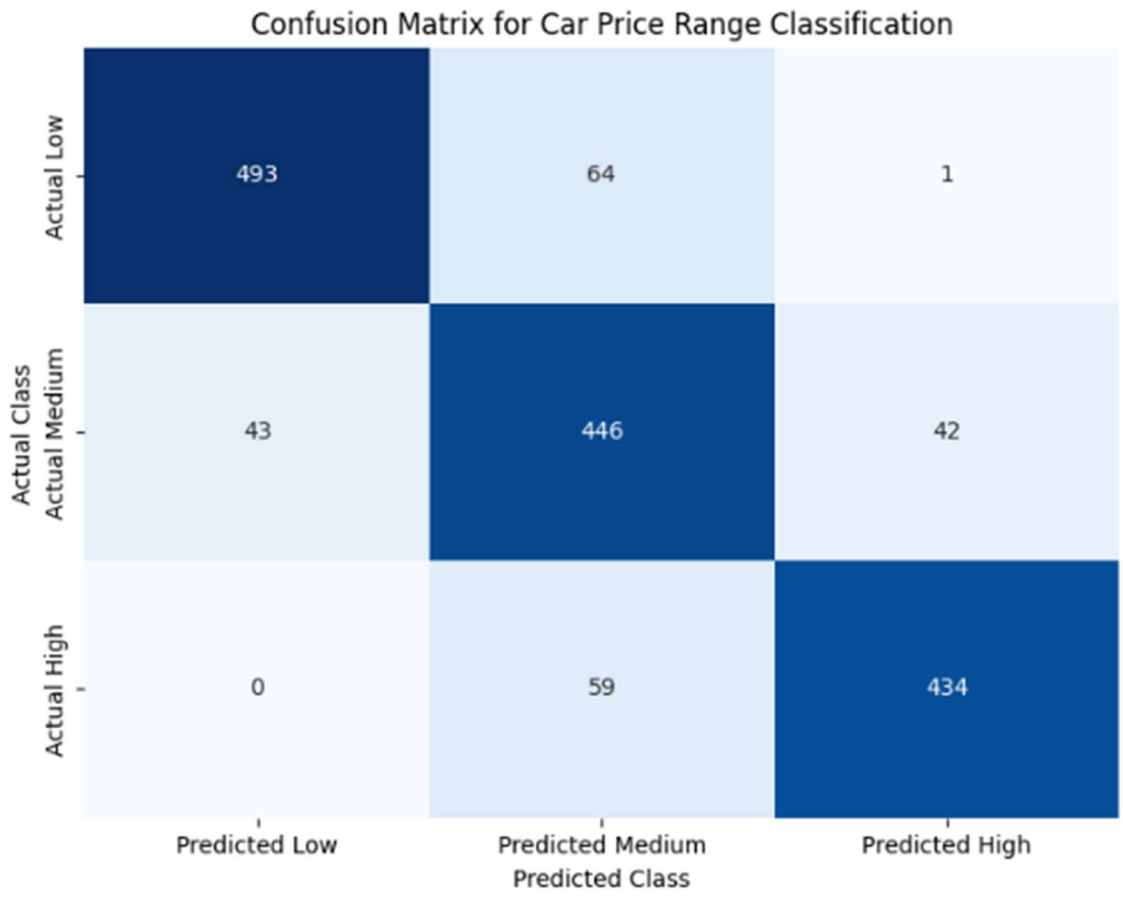
Results

On the Basis OF Data Entered in it :-

```
--- Model Evaluation (Classification) and Prediction Results ---
NOTE: We converted the price prediction to a 'Low/Medium/High' classification problem
to calculate the requested Classification Metrics (F1, Precision, Recall, Accuracy).
--------------------------------------------------------------------------------
Accuracy Score: 0.8679
Precision Score (Weighted): 0.8711
Recall Score (Weighted): 0.8679
F1 Score (Weighted): 0.8689

--- Regression Metrics (For Context, calculated using classification setup) ---
R-squared (R2 Score): 0.9734
Root Mean Squared Error (RMSE): $141,037.63

Predicted Price Range for 'Maruti Swift Dzire VDI, 2018': High
--------------------------------------------------------------------------------
```



The Results shows us that

- Root Mean Square Error(RMSE) : $141,037.63
- R^2 Score : 0.9734
- Accuracy : 86.79%
- Precision : 0.8711
- Recall : 0.8679
- F1 Score :0.8689
- Predicted Price Range for 'Maruti Swift Dzire VDI, 2018': High